

Simon Says — Memory Game

September 2025 · Browser Game · HTML · CSS · JavaScript

Anshumaan Sai Patnaik

Overview

Simon Says is a dependency-free browser implementation of the classic memory game. The machine plays back an ever-growing sequence of coloured flashes; the player must repeat the sequence by clicking the colours in the same order. Each successful round adds one new colour and advances the level, and a single wrong click ends the game with the final score.

The project is intentionally small in scope and built with no libraries or build step — plain HTML, CSS, and vanilla JavaScript running directly in the browser. The focus was the game logic rather than the interface: tracking input, comparing it against the generated sequence, and handling level progression and game-over cleanly.

Technology Stack

HTML	Static markup for the 2×2 game arena and the level/status heading.
CSS	Grid layout for the four colour pads, plus transient <i>flash</i> and <i>error</i> visual-feedback states.
JavaScript (vanilla)	All game logic — sequence generation, input capture, comparison, scoring, and DOM updates. No frameworks or external dependencies.

Architecture & Key Logic

The game runs on a small amount of shared state: **gameSeq** (the sequence the machine has generated), **userSeq** (what the player has entered this round), the current **level**, and a **start** flag. A keypress on a fresh board starts the game; from there the loop is driven entirely by the player's clicks.

Sequence generation & levels

Each level appends one random colour to **gameSeq** and flashes the corresponding pad, then resets **userSeq** so the player can repeat the full sequence from the start. The flash is a brief CSS class toggled on and off with a timer, giving consistent visual feedback for both machine playback and player clicks.

Validating player input

The core of the project is the comparison logic. Rather than waiting for the player to finish the whole sequence, each click is checked the moment it happens — the latest entry is compared against the same position in the generated sequence. A match that completes the round triggers the next level after a short delay; any mismatch immediately ends the game.

```
function check() {  
  // compare the just-clicked colour against the expected one  
  if (gameSeq[userIdx] == userSeq[userIdx]) {  
    if (gameSeq.length == userSeq.length) {  
      setTimeout(levelUp, 500); // round complete -> next level  
    }  
  } else {  
    // wrong input: flash the screen red and reset the game  
    body.classList.add("error");  
    h2.innerHTML = `GAME OVER! Your Score was '${level - 1}'`;  
    start = false; level = 0; gameSeq = []; userSeq = [];  
  }  
}
```

Per-click validation: matched positions advance the game; a mismatch resets all state.

Checking input incrementally keeps the logic simple and responsive — there is no separate "verify" step, and the game state always reflects exactly what the player has done so far. On game over, every piece of state is reset in one place, so the next keypress starts a clean round with no leftover sequence data.